Semantic MapReduce: An Auditable Operator Contract for LLM-Backed Reduction

Anonymous Author(s)

Abstract

Data engines define precise contracts for scans, joins, windows, and aggregation, but an LLM placed above those operators can still change analysis state through unaudited text. The missing contract specifies which records become evidence, which evidence may be fused, which model decisions are admissible, and what state remains after rejection. We present *Semantic MapReduce*, an operator algebra and runtime contract for reducing partitioned observations into evidence-linked hypotheses. Its data-facing operators produce normalized evidence and candidate groups; SemanticReduce produces a valid baseline; a model may implement only a bounded Edit in {Keep, Merge, Split, Abstain}; and system-owned Validate and ReportTrace bind evidence identifiers, enforce invariants, preserve fallback state, and record replay keys and state digests. We integrate the contract with workflow/stream execution, checkpointable runtime state, public telemetry replay, and an OpenAI-compatible endpoint. Across nine controlled workload families and ten seeds, a hint-aware incident reducer reaches 0.7796 mean F1 versus 0.0705/0.4916 for map/window baselines. A 27-run fault-injection matrix validates commit, reject, checkpoint restore, and deterministic replay in every run. In a real-online matrix over the same nine families and three seeds, bounded action validation raises mean F1 from 0.7801 to 0.8571 and support recall from 0.7810 to 0.8795, with zero invalid actions, invalid schemas, or fallbacks. These results support a systems conclusion: model-backed reduction should enter a runtime as a bounded, evidence-linked state transition, not as unconstrained generated state.

1 Introduction

MapReduce and its successors give data systems a precise local/global decomposition: data engines own partitioning, scans, joins, windows, and distributed scheduling, while reducers combine typed intermediate state [2, 3, 9, 18]. This contract works when the output unit and aggregation rule are known in advance. Operational analysis increasingly asks for a different global result: infer which shard-local symptoms form one incident, preserve the evidence for that hypothesis, and expose conflicts or uncertainty. The result is not another numeric aggregate but a semantic state transition over distributed evidence.

Attaching an LLM to a database, dashboard, or agent loop does not define that transition. Prompt text can silently choose evidence, fuse unrelated observations, omit required fields, or overwrite a valid result, while JSON mode constrains syntax but not evidence eligibility or recovery. This makes a prompt-centric design difficult to compose with runtimes and impossible to audit as an operator: neither the changed state nor the admissibility rule is explicit. A monolithic LLM system is also the wrong layer because storage, data movement, scheduling, and inference already belong to mature engines.

We therefore ask: *what operator and runtime contract can admit model judgments over partitioned evidence without delegating state integrity to the model?* Four requirements follow. Operator boundaries must carry typed evidence rather than prompt text; the model-visible decision must have a finite effect on runtime state; every accepted transition must preserve schema and provenance invariants; and rejection must leave a valid fallback state. These requirements distinguish an OS/PL contract—typed objects, state transitions, invariants, and recovery—from a prompt template.

Semantic MapReduce realizes this contract with the algebra in Figure 1. Shard, MapEvidence, Normalize, and GroupEvidence convert observations into bounded candidate groups. SemanticReduce computes a valid baseline H_0. An optional Edit proposes a transition Δ using only Keep, Merge, Split, or Abstain. System-owned Validate either commits the transition or preserves H_0; ReportTrace emits the hypothesis with the evidence and decisions needed to replay it. Rules, graph algorithms, or models may implement semantic policy, but only structured objects cross the contract.

The prototype maps this algebra onto a workflow/stream runtime without modifying the storage or serving engines. Dataflow stages implement the scale-facing operators; reducer plug-ins share the same candidate, hypothesis, and trace types; an OpenAI-compatible endpoint can supply Edit decisions; and the runtime owns evidence identifiers, edit assembly, validation, fallback, latency/token accounting, and trace capture. Thus model output cannot directly become report state. It is an untrusted proposal interpreted through the same commit/reject interface as deterministic and hybrid policies.

We evaluate the abstraction in four evidence tiers. A nine-family, ten-seed controlled matrix varies coverage, fragmentation, shared causes, concurrency, false correlation, partial evidence, overmerge, and edit admissibility under one schema and scorer. A public AIOps Challenge 2020 replay tests the evidence boundary on labeled metrics and exposes two detectable and two coverage-limited fault windows. A 27-run runtime matrix injects invalid edits and checkpoint recovery across every controlled family; all runs reject invalid

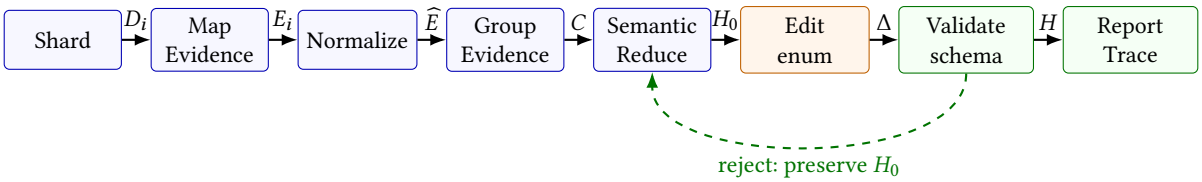

Figure 1. Semantic MapReduce is an operator and state-transition contract. Existing data runtimes can supply the scale-facing operators. A model may influence only the bounded Edit transition; system-owned Validate binds evidence, checks schema and consistency invariants, and preserves a valid pre-edit state on rejection. ReportTrace makes the committed execution auditable.

state, preserve the baseline, restore the committed digest, and replay deterministically. Finally, a single-accelerator real-online matrix runs the same nine families and three seeds: bounded action validation improves mean F1 from 0.7801 for the hybrid baseline to 0.8571 with zero invalid actions, invalid schemas, or fallbacks. The online result is mechanism evidence under one model and endpoint, not production or cross-model robustness.

This paper makes three contributions:

1. **Operator contract.** We define a typed algebra for evidence-linked semantic reduction and specify bounded edit, validation, provenance, and recovery invariants that separate model policy from runtime state integrity.
2. **Runtime integration.** We instantiate the algebra in a workflow/stream runtime that keeps edit assembly, validation, fallback, state digests, checkpoint recovery, accounting, and replayable traces system-owned.
3. **Workloads and evidence.** We organize nine reproducible contract-obligation families, add a public-telemetry evidence replay and a recovery fault matrix, and report these separately from the real-online mechanism study.

2 Motivation

Operational analysis often starts with numerical aggregates but ends with semantic judgment. In an accelerator-backed LLM serving platform, a dashboard may show that p95 latency increased in one region, decode utilization saturated, scheduler queue depth grew, and errors rose for some tenants. Each observation is measurable with conventional analytics; the harder question is whether these observations are one incident, several unrelated incidents, or noise. Conventional engines provide the substrate for scans, windows, joins, and stateful aggregation, but they do not normally expose first-class incident hypotheses, evidence objects, conflict handling, or explanation traces.

LLM-enabled data tools often expose a chat interface over a database, log store, vector index, or dashboard. This is useful for ad hoc exploration, but it hides important systems structure: which context was inspected, how evidence was compressed, when follow-up queries were issued, and why a final answer should be trusted. Large-scale analysis needs explicit partitioning, bounded local evidence, semantic reduction, and traceable reports.

The orchestration layer is the right place for this contract. It does not replace Spark, Flink, Ray, SQL engines, vector stores, or serving engines. Those systems own distributed execution, storage access, stream processing, indexing, model serving, and resource management. The orchestration layer expresses the LLM reasoning workflow as a dataflow/stream pipeline, coordinates semantic operators, and records intermediate evidence.

3 Problem Statement

We define large-scale LLM-augmented data analysis as follows. The input is a large collection of heterogeneous observations: events, logs, metrics, traces, documents, experiment outputs, and derived aggregates. The output is a set of structured insights, incident hypotheses, explanations, evidence chains, and possible actions. A correct system must manage both scale and semantics.

This problem exposes six system challenges:

- **Scale and partitioning.** Data must be split across shards while preserving enough local context to extract meaningful evidence.
- **Evidence compression.** Raw records cannot all be sent to a reducer or LLM. Map stages must summarize without losing the signals needed for global reasoning.
- **Semantic reduction.** The reducer must merge related symptoms, remove duplicates, handle conflicts, and produce hypotheses rather than only numeric aggregates.
- **Auditability.** Reports must link back to the evidence and workflow decisions that produced them.
- **Latency and cost.** LLM calls, if used, must be bounded and placed where semantic value justifies cost.
- **Safety boundaries.** The orchestration layer must avoid leaking unnecessary raw data and must make it possible to restrict what reaches external models.

These constraints motivate a system that treats LLM interpretation as a workflow over evidence objects, not as a final chat step after data processing.

4 Semantic MapReduce Operator Model

The central claim of Semantic MapReduce is that LLM-native analysis needs standard operators, analogous in spirit to `map` and `reduce`, but with contracts for evidence, bounded semantic edits, validation, and explanation. The abstraction is not an agent prompt wrapped around a database. It is an operator model in which observations become evidence, evidence becomes reducer candidates, reducer candidates become hypotheses, and every semantic edit is either validated or rejected before it affects the report.

Let D denote a collection of observations, $\mathcal{E}$ evidence objects, $\mathcal{C}$ reducer candidates, $\mathcal{H}$ hypotheses, Δ reducer edits, $\mathcal{T}$ workflow trace records, and $\mathcal{Y}$ the final report. Semantic MapReduce composes the following operators:

$$\mathcal{A} = \{\textsc{Keep}, \textsc{Merge}, \textsc{Split}, \textsc{Abstain}\} \tag{1}$$

$$\mathcal{S} = \mathsf{Shard}(D; \pi) \tag{2}$$

$$\mathcal{E} = \bigcup_{s \in \mathcal{S}} \mathsf{MapEvidence}(s; \phi) \tag{3}$$

$$\widehat{\mathcal{E}} = \mathsf{Normalize}(\mathcal{E}; \sigma) \tag{4}$$

$$\mathcal{C} = \mathsf{GroupEvidence}(\widehat{\mathcal{E}}; \gamma) \tag{5}$$

$$(\mathcal{H}_0, \mathcal{T}_R) = \mathsf{SemanticReduce}(\mathcal{C}; \rho) \tag{6}$$

$$\Delta = \mathsf{Edit}(\mathcal{C}, \mathcal{H}_0; a), \quad a \in \mathcal{A} \tag{7}$$

$$(\mathcal{H}, \mathcal{T}_V) = \mathsf{Validate}(\mathcal{H}_0, \Delta, \widehat{\mathcal{E}}; V) \tag{8}$$

$$(\mathcal{Y}, \mathcal{T}) = \mathsf{ReportTrace}(\mathcal{H}, \widehat{\mathcal{E}}, \mathcal{T}_R \cup \mathcal{T}_V; \tau). \tag{9}$$

The policies $\pi, \phi, \sigma, \gamma, \rho, V, \tau$ may be implemented by rules, queries, stream processors, graph algorithms, LLM calls, or hybrids. The important invariant is that only structured objects cross operator boundaries. Local findings are evidence objects rather than prompt text; reducer inputs are candidate groups rather than raw telemetry; model-backed edits are bounded actions rather than complete incident reports; and reports preserve evidence links rather than only natural-language conclusions.

This model separates the part a model may do from the part the system must own. SemanticReduce can be deterministic, graph-based, LLM-backed, or hybrid. When a model is used, the safer interface is often Edit: the model makes a bounded semantic decision such as whether two candidates should be merged, split, kept separate, or abstained from. The system then assembles the edit payload, binds evidence identifiers, checks the incident schema, verifies root and affected-service consistency, and falls back when validation fails. This is why the evaluated one-token reducer is not merely a prompt variant: it changes the reducer contract from "generate JSON hypotheses" to "choose a bounded action that the system can validate."

External data engines may implement the scale-facing operators, while LLM frameworks may implement semantic decisions or reporting. The orchestration layer composes them under one auditable evidence contract and owns the boundaries where data-plane outputs become semantic-control-plane decisions. The operators expose five contract surfaces: data engines provide shards, windows, summaries, or chunks; local findings become bounded evidence objects rather than prompt text; reducers may change strategy while preserving the incident schema and scorer; model-backed edits must pass evidence and schema validation; and ReportTrace ties every hypothesis to operator decisions, supporting evidence, and replay metadata.

Shard and map evidence. Each shard is processed locally. MapEvidence may compute statistics, identify anomalies, summarize logs, classify symptoms, or invoke an LLM over a bounded context. Its output is not arbitrary prose. It is an evidence object with fields such as service, tenant, region, metric, time window, observed value, baseline, severity, confidence, and source references.

Normalize, group, and semantic reduce. Normalize and GroupEvidence prepare evidence for global reasoning. SemanticReduce then merges evidence objects into higher-level hypotheses. This stage deduplicates adjacent windows, combines weak but consistent signals, separates unrelated incidents, handles contradictory evidence, and assigns confidence. The reducer may be deterministic, LLM-backed, or hybrid. Deterministic reducers are useful baselines because they are reproducible and cheap; LLM reducers are attractive when the merge requires domain language, causal reasoning, or flexible explanation.

Edit and validate. A model-backed reducer need not emit final hypotheses. It can instead implement Edit: a bounded decision over reducer candidates. In the evaluated action reducer, the model chooses only Keep, Merge, Split, or Abstain. Validate then checks that any proposed edit preserves evidence references, schema constraints, root and affected-service consistency, and the no-regression fallback policy. Invalid model text becomes Abstain; invalid edits are rejected; unsafe reducer outputs fall back to the deterministic or hybrid baseline.

Report and trace. ReportTrace turns hypotheses into operator-facing reports. The report should include the claim, supporting evidence, possible conflicting evidence, and suggested actions. It should also preserve the workflow trace so a user can inspect how the conclusion was formed.

Semantic MapReduce differs from an agent loop because its units of parallelism, evidence compression, reduction, and reporting are explicit. It also differs from ordinary stream processing: stream windows can compute features and alerts, but semantic reduction turns local findings into a global explanation.

Algorithm 1 makes the operator vocabulary operational. The important constraint is that SemanticReduce consumes compact, normalized, grouped evidence objects rather than raw telemetry, and that model-backed Edit decisions are validated before they affect $\mathcal{H}$. This lets the orchestration layer

Algorithm 1 Semantic MapReduce Workflow

Require: Observations D, policies $\pi, \phi, \sigma, \gamma, \rho, V, \tau$
Ensure: Evidence-linked incident report $\mathcal{Y}$ and workflow trace $\mathcal{T}$

$\mathcal{S} \leftarrow \textsc{Shard}(D; \pi)$ ▷ partition by service, region, tenant, or time
$\mathcal{E} \leftarrow \emptyset, \mathcal{T} \leftarrow \emptyset$
for all shard $s \in \mathcal{S}$ **do**
  $(E_s, T_s) \leftarrow \textsc{MapEvidence}(s; \phi)$
  $\mathcal{E} \leftarrow \mathcal{E} \cup E_s$
  $\mathcal{T} \leftarrow \mathcal{T} \cup T_s$
end for
$\widehat{\mathcal{E}} \leftarrow \textsc{Normalize}(\mathcal{E}; \sigma)$
$\mathcal{C} \leftarrow \textsc{GroupEvidence}(\widehat{\mathcal{E}}; \gamma)$
$(\mathcal{H}_0, T_R) \leftarrow \textsc{SemanticReduce}(\mathcal{C}; \rho)$
$(\Delta, T_E) \leftarrow \textsc{Edit}(\mathcal{C}, \mathcal{H}_0)$ ▷ optional bounded edit
$(\mathcal{H}, T_V) \leftarrow \textsc{Validate}(\mathcal{H}_0, \Delta, \widehat{\mathcal{E}}; V)$
$\mathcal{T} \leftarrow \mathcal{T} \cup T_R$
$\mathcal{T} \leftarrow \mathcal{T} \cup T_E \cup T_V$
$\mathcal{Y} \leftarrow \textsc{ReportTrace}(\mathcal{H}, \widehat{\mathcal{E}}, \mathcal{T}; \tau)$
return $(\mathcal{Y}, \mathcal{T})$

place expensive or policy-sensitive LLM calls at semantic boundaries instead of at every scan or filter operation.

5 Runtime and Orchestration Design

The prototype keeps the execution boundary deliberately narrow. Storage, joins, stream windows, distributed scheduling, and model serving remain delegated to existing engines; the orchestration layer owns the semantic control plane: operator composition, evidence schemas, reducer selection, validation, and trace capture. This is the difference from simple stitching. A script can query a data engine and then call a model, but it does not define which evidence objects cross the boundary, how duplicate local alerts become one hypothesis, or how a report links back to reducer decisions.

The workflow graph is the runtime representation of this contract. It composes familiar stream/dataflow operators–map, filter, flatmap, key grouping, and connection–but assigns them semantic roles. Sharded maps implement MapEvidence; keyed stages implement GroupEvidence; downstream stages invoke SemanticReduce, optional Edit, Validate, and ReportTrace. Upstream systems may supply partitions, windows, feature tables, retrieved documents, or trace chunks. Downstream tools consume structured incidents and evidence-linked reports. The orchestration layer therefore sits above Spark, Flink, Ray, databases, vector stores, and serving endpoints rather than replacing them.

Evidence objects are the bridge between data systems and LLM reasoning. They include shard and time-window identifiers, service/region/metric fields, local baseline statistics, anomaly type, severity, confidence, and source references. Reducers consume grouped evidence and emit the same incident schema plus a reducer trace. The evaluated implementation includes map-only and window baselines, deterministic and hybrid semantic reducers, free-form LLM reducers, and action-constrained reducers under one scorer. Changing the reducer therefore does not require rewriting the generator, mapper, report schema, or benchmark matrix. The trace records reducer metadata, detected incidents, matched and missed incident IDs, evidence summaries, failure classes, run manifests, and token estimates, so false positives and false negatives can be attributed to evidence extraction, grouping, editing, validation, or thresholding rather than to an opaque final answer.

6 Workload and Provenance

6.1 Scenario

The workload models telemetry from an NPU-backed LLM serving platform. Services include `prefill`, `decode`, `kv-cache`, `scheduler`, `router`, and `embedding`. Events include service, tenant, region, timestamp, latency, error flag, NPU utilization, queue depth, and generated metadata. The generator injects hidden incidents of three types:

- **Latency spike:** one service experiences elevated latency over a time window.
- **NPU saturation:** utilization increases and may affect queueing or latency.
- **Queue backlog:** scheduler or routing queues grow and create downstream symptoms.

The task is to recover injected incidents from event streams. A detected incident is useful only if it links to the supporting evidence and matches the hidden ground truth by type, service, region, and time.

Why this workload exercises the system. The workload is designed to stress the orchestration layer rather than raw storage throughput. It has enough structure to require semantic reasoning: incidents are injected across services, regions, and time windows, while local symptoms appear as latency, error, utilization, or queue-depth changes. A single shard can observe symptoms, but the global conclusion depends on how those symptoms are merged. This matches the Semantic MapReduce pattern.

The workload also forces the system to preserve evidence. If the reducer reports a latency spike, the report must include which shard-level candidates supported the decision. If it misses a saturation incident, the report should expose the missed incident ID. This is why the workload reports both detected and injected incidents, not only aggregate F1. The goal is not merely to score a classifier; it is to evaluate an auditable analysis workflow.

Finally, the workload is intentionally reproducible. Seeds, event counts, shard counts, top-k values, reducer names, incident reports, missed incidents, reducer traces, workflow

traces, run manifests, failure classes, and token/cost accounting fields are recorded. The same matrix can be rerun when a reducer changes. This gives the prototype a benchmark harness for model-backed reducer experiments without making the current paper depend on a live model endpoint.

The semantic-merge workloads in Table 1 separate positive edits from no-op and adversarial obligations. This matters because an edit protocol can appear strong if every case rewards merging. Sanity and cascade families test ordinary incident formation; concurrent and false-correlation families require separation; partial evidence distinguishes a map-coverage limit from a reducer error; and the three ambiguity families force merge, split, or abstain decisions after coverage is complete. All nine families use the same evidence schema and scorer in both the ten-seed derived matrix and the three-seed real-online matrix; the ambiguity subset is retained only as a mechanism diagnostic.

6.2 Pipeline and Implementation

The workload instantiates the operator model directly. Shard partitions telemetry by shard count, service, region, and time. MapEvidence and Normalize turn local anomalies into scored evidence objects, and GroupEvidence clusters candidates by incident type, service, region, and temporal overlap. SemanticReduce then emits hypotheses through map-only, window, deterministic, hybrid, stubbed, or model-backed reducers. Candidate-level and one-token action reducers exercise Edit and Validate: the model proposes bounded actions, while the system assembles evidence-linked edits, checks schema and service invariants, and falls back when required. ReportTrace scores hypotheses, records matched and missed incidents, classifies reducer failures, and emits per-run reports.

The workload reports precision, recall, F1, throughput, map latency, reduce latency, detected incidents, injected incidents, and missed incidents. Reports also include seed, top-k, reducer name, and incident matching metadata.

The implementation has three components. The generator creates synthetic telemetry and hidden incidents. The map stage implements Shard, MapEvidence, and Normalize by partitioning events, computing local summaries, and producing anomaly candidates. The reducer implements GroupEvidence and SemanticReduce over shard summaries. The default reducer is deterministic: it clusters candidates using incident type, service, region, and temporal overlap. The map-only and window-aggregation reducers are diagnostic baselines that approximate alert-only and windowed-analytics workflows without incident-level semantic reduction. The `llm-stub` reducer is an interface placeholder that exercises the same resolver and report path while delegating to the deterministic reducer.

The report schema is deliberately richer than the minimum needed for precision and recall. It includes the map policy, reducer name, seed, top-k, injected incidents, detected incidents, matched incident IDs, and missed incidents. This enables per-run diagnosis of operator behavior. When a run misses an incident, the missed incident ID is preserved in the report rather than disappearing into the aggregate recall number; when a map policy changes, the same reducer and scorer can be replayed over the new evidence stream.

The workload is designed to make reducer behavior reproducible without placing environment-management details in the paper body. Each experiment records the workload scale, shard count, seed, reducer, top-k budget, detected incidents, missed incidents, and latency measurements. This provenance is sufficient to distinguish algorithmic changes in the reducer from changes in input generation or scoring. The accompanying artifact contains the implementation and per-run records needed to rerun the study.

7 Evaluation

7.1 Setup and Scope

Table 2 lists the reproduced scales and representative single-seed runs used for scale/timing sanity checks. These rows are not the only quality evidence; the paper uses the ten-seed matrix for reducer and map-policy conclusions. They show a useful systems property: doubling events and shards roughly doubles map time, while the deterministic reducer still runs over compact evidence in less than one millisecond.

The evaluation follows the operator contract rather than a single leaderboard. Q1 asks whether the workload exposes the right semantic object: incident hypotheses rather than alert fragments. Q2 asks whether MapEvidence surfaces enough evidence for any reducer, using coverage gates. Q3 asks whether reducer variants improve under a fixed generator, evidence schema, scorer, and report format. Q4 asks whether adjacent frameworks are substrates or whether they supply the missing semantic-reduction contract. Q5 asks whether a live model can be admitted as bounded Edit, rather than free-form SemanticReduce. This evidence ladder prevents the F1 tables from becoming a generic ML benchmark: precision, recall, and F1 check whether evidence objects are preserved, grouped, and reduced into the right incident unit. Unless explicitly labeled real-online, F1 and seed-matrix results in this section are derived artifacts from recorded workload reports; the real-online rows support live reducer-interface, validation, latency, and token-accounting claims only. The experiments are not tuned cluster deployments of Spark, Flink, Ray, LangGraph, LlamaIndex, or AutoGen.

The artifact also includes a separate shared-workload probe from a pinned workload submodule. Across seeds 7, 11, and 13, the probe generates 23 canonical LLM-serving cases per seed and 1,232 supported requests per seed, covering session-affine, RAG, long-context, tool-agent, coding-assistant, dynamic-corpus, memory-reuse, and preemption/resume surfaces. We use this probe to record workload-source provenance and to keep prototype-specific Semantic MapReduce stress cases

Table 1. Workload coverage is organized by reducer-contract obligation. All nine families have ten-seed derived matrices, three-seed runtime fault injection, and three-seed real-online action evidence.

Family	Evidence condition	Required reducer behavior	Current evidence tier
Single service	local, complete	preserve a trivial incident	derived + real-online
Cascade	cross-service, complete	fuse downstream symptoms	derived + real-online
Shared bottleneck	shared cause, complete	recover one common cause	derived + real-online
Concurrent	overlapping independent incidents	keep incident units separate	derived + real-online
False correlation	correlated but unrelated symptoms	reject a spurious merge	derived + real-online
Partial evidence	root observation absent	expose coverage/inference boundary	derived + real-online
Ambiguous overmerge	candidate too coarse	split or safely abstain	derived + real-online
Disconnected merge	one incident fragmented by grouping	accept evidence-preserving merges	derived + real-online
Temporal split	one incident spans candidate windows	merge without losing temporal support	derived + real-online

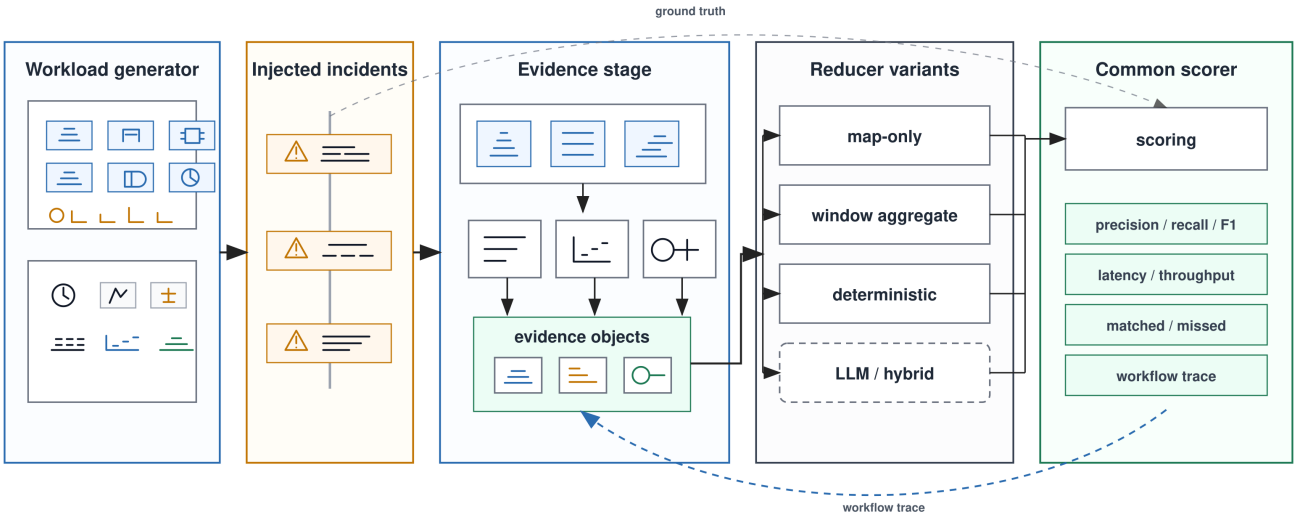

Figure 2. Evaluation harness for the large-scale analysis workload. The same generated telemetry and injected incidents feed the evidence stage, reducer variants, and common scorer, so quality differences come from the semantic reduction contract rather than input drift. Solid reducer rows correspond to the deterministic matrix; the dashed model-backed branch is evaluated in the nine-family real-online semantic-merge matrix.

separate from general LLM-serving scenario catalogs. It is not used as evidence that a reducer improves incident quality.

Table 3 gives the shared workload side of the artifact. The corresponding Semantic MapReduce quality experiments remain repo-local because they require injected incident labels, evidence objects, reducer variants, and a common scorer. This separation prevents a common benchmark mistake: using a general workload catalog as if it were already a reducer-quality benchmark.

7.2 Reducer Quality

The aggregate results in Table 4 show why incident-level reduction is the relevant operation. Map-only alerts emit duplicate shard-local candidates and consume the top-k budget before all incidents are represented. Window aggregation improves recall but still reports multiple windows for one incident. The incident reducer keeps recall at 0.9750 while reducing average detections from 7.05 to 4.15, raising F1 from 0.7129 to 0.9579. In this paper, F1 is therefore a contract-level metric: it is high only when the workflow preserves

Table 2. Workload scales and representative seed-7 tail-aware deterministic runs. The full matrix repeats these two scales across ten seeds, map policies, and reducer regimes. Both representative runs detect all four injected incidents; their timing shows that MapEvidence dominates while reduction over compact evidence remains sub-millisecond. Evidence: derived artifact.

Scale	Top-k	P/R/F1	Thru. k ev/s	Map/Red. ms	Det.
50K/16	12	1.00/1.00/1.00	105.8	72.94/0.35	4/4
100K/32	16	1.00/1.00/1.00	101.6	169.61/0.31	4/4

Table 3. Shared workload-source probe from the pinned workload submodule. This table records scenario coverage and provenance for the artifact boundary; Semantic MapReduce quality is measured only on workloads with injected incident labels and a common scorer.

Field	Value
Submodule commit	`79ed8e3`
Seeds	7, 11, 13
Generated cases / seed	23
Skipped boundary cases	2
Supported requests / seed	1,232
Example long-context prompt	4,442 tokens
Example RAG prompt	2,974 tokens
Evidence label	derived artifact

Table 4. Reducer comparison with the tail-aware map policy across 20 runs. Map-only and window-aggregate are diagnostic baselines for alert-style and windowed-analytics workflows; the incident reducer improves F1 by changing the output unit from alert fragments to incident hypotheses. Evidence: derived artifact.

Reducer	Precision	Recall	F1	Detections
Map-only	0.1990	0.6875	0.3075	14.00
Window-aggregate	0.5696	0.9750	0.7129	7.05
Incident reducer	0.9500	0.9750	0.9579	4.15
`llm-stub`	0.9500	0.9750	0.9579	4.15

enough evidence and emits the correct incident unit with traceable matches. This is the measured advantage in the current workload: semantic reduction changes the unit of output from alert fragments to incident hypotheses.

Table 5 adds the semantic-merge suite. A root cause in one service can produce symptoms in dependent services, so the scorer requires the hypothesis to recover the root service, region, overlapping time, and enough affected services. The nine-family aggregate is intentionally harder than

Table 5. Semantic-merge suite across nine contract-obligation families and ten seeds (90 runs per reducer). Evidence: derived artifact.

Reducer	Precision	Recall	F1	Detections
Map-only	0.0519	0.1111	0.0705	13.97
Service-local	0.0533	0.1111	0.0717	13.27
Window-aggregate	0.3779	0.7444	0.4916	8.17
Semantic-graph	0.6485	0.6639	0.6435	4.97
Hybrid-hint	0.7585	0.8556	0.7796	4.97
`llm-stub`	0.6485	0.6639	0.6435	4.97

Table 6. MapEvidence policy ablation for the deterministic incident reducer. The tail-aware policy preserves high-p95 NPU saturation evidence; mean-only corresponds to the earlier mean-utilization rule. Evidence: derived artifact.

Config	Policy	P	R	F1	Missed
50K/16/12	Mean	0.9200	0.9750	0.9413	1/10
50K/16/12	Tail	0.9200	0.9750	0.9413	1/10
100K/32/16	Mean	0.9750	0.9250	0.9464	3/10
100K/32/16	Tail	0.9800	0.9750	0.9746	1/10

the earlier seven-family diagnostic: disconnected and temporally fragmented incidents reduce the semantic-graph aggregate to 0.6435 F1, while the hint-aware reducer reaches 0.7796. The single-service sanity case still lets map-local policies succeed, so the benchmark is not constructed to make them always fail. Each family changes one semantic contract obligation while holding the generator, evidence schema, and scorer fixed. In *partial-evidence*, the hint-aware reducer raises F1 from 0.5761 to 0.8878. In *ambiguous-overmerge*, complete evidence can still collapse into one candidate and therefore motivates a bounded split; disconnected and temporal families require evidence-preserving merges. These are mechanism obligations, not generic model-failure labels.

Table 6 is the most important diagnostic result. The larger workload exposes a failure that is not solved by changing the reducer alone: with mean-only NPU detection, three of ten 100K runs miss at least one incident. The missed cases include NPU saturation windows whose mean utilization is diluted by surrounding normal events, even though their tail utilization is high. Adding a stricter p95 NPU signal to MapEvidence reduces missed runs from three to one and raises 100K F1 from 0.9464 to 0.9746.

Table 7 adds a stronger guardrail: before comparing reducers, the system should measure whether the map stage surfaced the injected incidents as evidence. A later trace-guided pass exposed a second evidence failure: low-baseline services such as routers can suffer real latency spikes whose absolute p95 remains small and whose window mean rises with the incident. A baseline-aware map policy compares

Table 7. Coverage gate over ten seeds. A reducer-quality comparison is meaningful only when injected incidents enter the evidence objects. Baseline-aware evidence reaches full coverage for 10K and larger settings in this workload; the 2K tail-aware setting does not. Evidence: derived artifact.

Config	Policy	Cov. mean	Cov. min	Cand. mean
2K/4/8	Tail-aware	0.3000	0.0000	1.9
2K/4/8	Base-aware	0.7750	0.5000	5.1
10K/8/12	Tail-aware	0.9250	0.7500	18.2
10K/8/12	Base-aware	1.0000	1.0000	39.5
50K/16/12	Tail-aware	0.9750	0.7500	48.6
50K/16/12	Base-aware	1.0000	1.0000	97.7
100K/32/16	Tail-aware	0.9750	0.7500	47.5
100K/32/16	Base-aware	1.0000	1.0000	112.0

each service/region window against a robust shard-local latency baseline. On the same two-size, ten-seed matrix, this policy raises the deterministic incident reducer to 1.0000 mean precision, recall, and F1, while map-only remains at 0.3500 F1 and window aggregation at 0.6366 F1. This result strengthens the operator argument: once map, evidence, and reduce are separate contracts, a failure can be localized to the evidence policy rather than vaguely attributed to "the LLM" or "the reducer."

The ablation also prevents overclaiming. Tail-aware and baseline-aware evidence policies are workload policies, not universal detectors. Baseline-aware evidence increases the number of map candidates, so it must be paired with incident-level reduction; otherwise map-only precision falls sharply. A stronger system would learn or calibrate the evidence policy from real traces.

7.3 Substrate and Overhead Checks

Local runtime checks show that the map stage dominates because it scans and summarizes event shards, while the deterministic reduce stage runs over compact candidates. Adjacent-framework probes with Ray-local, LangGraph-local, and LlamaIndex-docstore wrappers obtain the same F1 as the native path when they are given the same evidence objects, reducer, scorer, and report format. These probes are not leaderboards over those systems. They support the narrower systems claim that existing substrates can carry the computation, but this workload still needs the evidence-linked reducer contract and audit trail.

Table 8 isolates runtime behavior from model quality. For each family/seed, the harness materializes a valid baseline in a checkpointable shared-state service, validates a legal transition, injects an edit with missing candidate evidence, and invokes checkpoint recovery. The runtime records the candidate, evidence, committed-state digests, commit/reject outcome, service revision, and replay identifier. All invalid edits preserve the canonical baseline, and all recovered and replayed states match the committed digest. This is not a claim of distributed exactly-once fault tolerance: it establishes that the implemented operator boundary has an executable recovery contract rather than relying on prompt compliance.

Table 8. Runtime-contract fault matrix: nine families × three seeds. Every row injects an invalid edit and checkpoint recovery. Evidence: derived artifact.

Invariant	Passes / runs
Valid transition commits	27 / 27
Invalid edit rejected	27 / 27
Baseline state preserved	27 / 27
Checkpoint digest restored	27 / 27
Trace replay deterministic	27 / 27

Table 9. AIOps Challenge 2020 May-29 public replay. Four official fault labels are paired with eight same-object negative windows. Evidence: replay.

Field	Result
Labeled / negative windows	4 / 8
Detected labeled windows	2 / 4
False-positive windows	0 / 8
Precision / recall / F1	1.000 / 0.500 / 0.667
Raw rows republished	no

Table 9 replaces source availability with an actual public-data replay at the evidence boundary. The runner reads the official daily ZIP in place, maps Docker/Oracle metrics to typed evidence using a fixed median/MAD gate, and archives only source references, evidence identifiers, scores, and provenance. Two CPU-fault windows are detected without false positives; a weak CPU shift and a window with insufficient samples are missed. We retain these misses because they expose the intended boundary: SemanticReduce cannot repair absent or weak MapEvidence. This replay broadens telemetry provenance, but it does not carry incident-group labels and therefore is not used to claim model-reducer quality.

7.4 Real-Online Readiness

The real-online runs attach the same reducer contract to a live OpenAI-compatible endpoint. A smoke run served a 7B instruction-tuned model on one accelerator and completed 8/8 measured streaming requests after warmup, with mean TTFT 65.68 ms, mean TPOT 19.79 ms, and 45.71 output tokens/s. These numbers validate endpoint integration and measurement plumbing rather than SOTA serving performance. Earlier same-seed reducer probes exposed two preconditions for meaningful live LLM comparison. First, if map evidence misses incidents, no downstream reducer

can recover them: a 2K/4-shard smoke produced F1 0.4000 because three injected incidents had zero overlapping map candidates. Second, asking a model to emit full reducer JSON is the wrong interface: full-evidence prompting returned legal JSON while losing incident units, and free-form candidate editors sometimes produced invalid structure or unsafe edits. The final live probe therefore focuses on the constrained Edit+Validate interface in Table 10. This makes the live path an admission test for a model-backed operator: a model action is useful only if it clears coverage, schema, evidence-reference, and fallback checks under the same scorer.

A follow-up pairwise edit probe narrows the interface further. Instead of asking the model to rewrite complete candidates, the reducer proposes short candidate pairs. The free-form version asks for a JSON decision list; the action version restricts each answer to `KEEP`, `MERGE`, `SPLIT`, or `ABSTAIN`, and the runtime constructs the edit payload. Table 10 expands the earlier hardcase probe to all nine families. The action path records zero invalid actions, invalid schemas, or fallbacks, raises pooled F1 from 0.7801 to 0.8571, and improves support recall from 0.7810 to 0.8795. It averages 122.61 ms and 203 tokens per run; free-form validated decisions average 3213.27 ms and 312 tokens while reaching 0.7896 F1. On the hardcase subset, action validation still reaches F1 1.0 on disconnected merge, 0.8/1.0/1.0 on temporal fragmentation, and preserves the 0.8571 baseline on overmerge. The ordinary/no-op families do not regress. This is one model/endpoint with controlled telemetry, but it shows that the bounded interface is both more effective and substantially cheaper than asking the same endpoint to generate decision JSON.

The benchmark records more than a summary score: configuration, reducer name, injected and detected incidents, matched and missed incident IDs, evidence support, latency, token estimates, and failure classes. In the 2K live smoke, for example, three injected incidents had zero overlapping map candidates; the low F1 therefore diagnoses a coverage failure, not a reducer-quality result.

The current online table is a point estimate, not a stability or frontier result. The artifact harness supports distinct repeated samples, retains each bounded-action response and provider usage envelope, and aggregates within-case F1 variance, exact action agreement, reducer latency, and measured tokens when the endpoint reports them (otherwise explicitly labeled character estimates). A clean repeated-sampling and candidate-budget sweep is a submission-freeze requirement. It is not reported here because the test credential requires post-exposure rotation; neither deterministic repetition nor derived aggregation is substituted for real-online model samples. A second controlled endpoint would strengthen external validity but is not part of the supported claim.

7.5 Lessons

The evaluation gives three lessons. First, the workload is not merely a classifier benchmark. Precision measures whether the reducer avoids inventing incidents from noisy evidence; recall measures whether the map and reduce stages preserve weak but real incidents; the missed-incident list shows which operator boundary failed. Second, the incident reducer improves quality mainly by changing the unit of output from windows to hypotheses. The output count drops toward the four injected incidents while recall remains high. Third, the map policy matters as much as the reducer: if a saturation event never becomes evidence, no downstream LLM can recover it without going back to raw telemetry.

These lessons define the role of an LLM reducer as a constrained semantic operator rather than a replacement for the pipeline. A model should not be rewarded for basic scanning, thresholding, or duplicate removal that deterministic operators already handle. Its useful role is to adjudicate ambiguous evidence groups, use service relationships to reject false positives, explain why symptoms belong together, and produce operator-facing reports from bounded evidence objects. The implemented online path exercises this contract and shows why validation is part of the system design: accepted model edits must preserve schema, evidence references, root/affected-service consistency, and cost visibility.

8 Open System Challenges

Semantic MapReduce separates the semantic control plane from data and serving planes. Data engines retain scans, joins, windows, and distributed execution; serving engines retain inference. The orchestration layer owns evidence eligibility, reducer policy, hypothesis provenance, validation, recovery metadata, and trace replay. The evaluated mechanism needs no new kernels or scheduler semantics. The public replay reduces, but does not eliminate, the external-validity gap: it covers real labeled metrics at the evidence boundary, not production incident grouping. We additionally replay the bounded commit/reject/replay contract over its detected public windows, but lack of incident-group labels still prevents scoring merge/split quality. Remaining challenges are broader data-plane attachment, public or production incident-group labels, cross-model and completed repeated-sampling studies, and serving-level constrained decoding only when a validator needs it. Evaluation should continue to establish coverage before reducer quality and live cost.

9 Auditability and Trust Boundary

Auditability is the reason to make semantic reduction an operator contract. A hypothesis is trusted because it replays to eligible evidence, reducer candidates, accepted edits, validator outcomes, and failure classes—not because its prose is fluent. The model therefore receives a bounded Edit over candidates that already cite evidence. The system constructs

Table 10. One-token pairwise action probe on one NPU endpoint across nine families and three seeds (27 runs per reducer). The action reducer restricts model output to an enum and lets the system assemble validated edits. Evidence: real-online; one model/endpoint.

Reducer	Precision	Recall	F1	Support rec.	Reduce ms	Tokens	Edits	Invalid act.	Invalid sch.	Fallback
Hybrid hint	0.7692	0.8426	0.7801	0.7810	0.26	0.0	0.0	0	0	0
Validated pairwise	0.8065	0.8056	0.7896	0.8116	3213.27	312.1	0.63	0	0	0
Validated action	0.8852	0.8426	0.8571	0.8795	122.61	203.1	0.81	0	0	0

the payload, checks schema and references, verifies root/affected-service consistency, records token and latency cost, and retains the pre-edit hypothesis on rejection. The live free-form/action contrast exercises this boundary directly: one-token output turns generation uncertainty into a checkable decision.

Three invariants make the contract portable. The *provenance invariant* requires every hypothesis to cite MapEvidence identifiers; generated rationale alone cannot create support. The *edit invariant* represents each post-grouping change as `KEEP`, `MERGE`, `SPLIT`, or `ABSTAIN` over existing candidates. The *recovery invariant* leaves a valid pre-edit hypothesis after rejection and restores committed state by digest after checkpoint recovery. Table 8 executes these state obligations. They are stronger than JSON mode, which says nothing about evidence eligibility, the modified candidate, or recovery.

The common trace also makes negative results diagnostic. A recall loss can be classified as missing map coverage or ignored covered evidence; a precision loss as noisy evidence or an over-merge; zero accepted edits and fallback reveal when the model contributed nothing. Reducer policy may change from deterministic to hybrid or learned without changing the scorer or report interface. The artifact records coverage, matched/missed incidents, support, accepted edits, invalid actions/schemas, fallback, tokens, latency, and failure classes as contract fields. This is an audit surface, not a claim of a complete security model.

10 Related Work

Execution substrates. MapReduce, Spark, and Flink provide the substrate for large-scale batch and stream analytics, and Ray provides a general distributed runtime for AI applications [2, 3, 9, 18]. Semantic MapReduce borrows the local/global decomposition but changes the reducer contract from fixed aggregation to semantic evidence fusion. These systems can execute or host parts of the pipeline; they do not by themselves define the evidence object, semantic edit, validation, and trace contract studied here.

LLM data and workflow systems. LOTUS and Palimpzest expose LLM calls as declarative or optimizable data-processing operators [7, 12]. LangGraph, LlamaIndex, and AutoGen provide useful abstractions for agents, retrieval, tool use, and multi-agent workflows [6, 8, 16]. Semantic MapReduce is complementary: it focuses less on query optimization or agent control flow alone and more on workflow/stream orchestration, incident-level semantic reduction, and evidence-linked audit traces under one scorer.

Serving and observability. vLLM shows how memory management and scheduling shape LLM serving performance [5]. DeathStarBench, FIRM, the Illinois trace dataset, OpenTelemetry Demo, AIOps Challenge 2020, and LO2 provide service graphs, metrics, logs, traces, and fault labels [1, 4, 10, 11, 13, 14]. Root-cause localization systems such as CloudRanger and MicroRank exploit service dependencies and trace contrasts [15, 17]. We replay AIOps 2020 at the evidence boundary, but retain controlled incident-group workloads because its public labels do not directly score our merge/split contract.

Data+AI platforms. Databricks, Snowflake Cortex, and BigQuery ML integrate analytics engines with machine learning and LLM functionality. They are important reference points, but systematic product comparison is outside this paper. The distinction we claim is narrower: an open workflow/runtime layer can sit above multiple engines and make semantic reducer behavior auditable through standard evidence, validation, and trace operators.

Positioning summary. The closest intellectual ancestor is MapReduce itself: local processing followed by global reduction. The closest practical neighbors are stream processors, distributed AI runtimes, LLM workflow frameworks, RAG/data frameworks, and data+AI platforms. The claim is not greater scale, feature completeness, or generality; it is that LLM-native analysis needs standard evidence operators, semantic reduction, evidence-linked reporting, and workflow traces under one reproducible contract.

11 Conclusion

Large-scale LLM-augmented analysis needs more than fast scans and more than chat over data. Semantic MapReduce extends map/reduce from fixed aggregation to evidence fusion: partition observations, compress evidence, reduce hypotheses, and preserve executable audit and recovery state. Nine controlled obligation families, a public telemetry replay, and a 27-run recovery matrix distinguish evidence coverage, reducer policy, and runtime integrity. The live experiment exposes the remaining systems requirement: model edits must be constrained to verifiable actions, schema-validated,

evidence-linked, and reversible before they become committed state. Model-backed reduction therefore needs an operator boundary that can admit, validate, abstain, recover, and replay.

This reframes the LLM as a bounded semantic decision procedure inside an auditable reducer. The system owns evidence eligibility, candidate construction, validation, fallback, and trace replay; reducer edits change workflow state only through checkable actions.

Acknowledgments

Generative AI tools were used to assist with drafting, editing, and code generation. The authors reviewed and validated the resulting text, code, experiments, and claims.

References

[1] Alexander Bakhtin, Jesse Nyyssölä, Yuqing Wang, Noman Ahmad, Ke Ping, Matteo Esposito, Mika Mäntylä, and Davide Taibi. 2025. LO2: Microservice API Anomaly Dataset of Logs and Metrics. In *Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering*. https://arxiv.org/abs/2504.12067

[2] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* (2015). https://sites.computer.org/debull/A15dec/p28.pdf

[3] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*. https://www.usenix.org/conference/osdi-04/mapreduce-simplified-data-processing-large-clusters

[4] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*. https://doi.org/10.1145/3293882.3339668

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. https://doi.org/10.1145/3600006.3613165

[6] LangChain. 2026. LangGraph: Build Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph.

[7] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. arXiv:2405.14696 [cs.CL]

[8] LlamaIndex. 2026. LlamaIndex: Data Framework for LLM Applications. https://github.com/run-llama/llama_index.

[9] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*. https://www.usenix.org/conference/osdi18/presentation/moritz

[10] NetManAIOps. 2020. AIOps Challenge 2020 Data. https://github.com/NetManAIOps/AIOps-Challenge-2020-Data.

[11] OpenTelemetry. 2026. OpenTelemetry Demo Architecture. https://opentelemetry.io/docs/demo/architecture/.

[12] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. arXiv:2407.11418 [cs.DB]

[13] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*. https://www.usenix.org/conference/osdi20/presentation/qiu

[14] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. Pre-processed Tracing Data for Popular Microservice Benchmarks. Illinois Data Bank. https://databank.illinois.edu/datasets/IDB-6738796.

[15] Pengfei Wang, Jingmin Xu, Meng Ma, Wei Lin, Dan Pan, Yuan Wang, and Pengfei Chen. 2018. CloudRanger: Root Cause Identification for Cloud Native Systems. In *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. https://doi.org/10.1109/CCGRID.2018.00076

[16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Adam White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]

[17] Guangba Yu, Pengfei Chen, Zicheng Li, Haifeng Zheng, Xusheng Lin, and Xiaofan Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *Proceedings of The Web Conference*. https://doi.org/10.1145/3442381.3449905

[18] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation*. https://doi.org/10.5555/2228298.2228301